Rate Limiting & Duplicate Submission Remediation

Target: `crm-api-stage.crusource.com` — `POST /api/v1/leads`

Dear Advaith and Arun,

Thank you for your time, it was truly pleasant to have a meeting with you guys. I have prepared the document as a reference guide to assist with implementing rate limiting. Please treat it as a helpful reference rather than a strict playbook; the specifics will depend on your architecture and deployment model. I've put it together to the best of my knowledge, and I welcome any corrections or clarifications from your side.

Recap

We sent 50 identical requests to the lead-creation endpoint at the same time, using Burp Suite. Every single request succeeded and created a new lead. Nothing was blocked or slowed down. This means the system has **no rate limiting**, so nothing stops someone from sending thousands of requests in a short time.

Important Note – While looking at one of those requests, I also noticed the data being sent includes a field called `"allow_duplicate": true`. This suggests the app already has logic that checks for duplicate leads, but that check is not initiated on the server side. That's a separate issue from rate limiting but crucial to investigate.

Why This Matters

- **Fake data piles up.** Every number on the dashboards (pipeline value, lead counts, "Closed Won" totals) becomes unreliable once duplicates mix in.
- **The database gets heavier than it should.** More junk rows mean slower searches and reports for real users.
- **Someone could abuse this on purpose.** Without limits, a script could create thousands of fake leads in minutes – a cheap way to disrupt the system or bury real leads in noise.
- **Servers can be exhausted.** As a large amount of data needs a large amount of processing, the server resources and the infrastructure surrounding it can be exhausted and taken down by a simple DoS attack.

Step-by-Step Fix Recommendation:

Step 1: Stop trusting the "allow_duplicate" flag

This is the most straightforward fix but crucial, so it should be done first. Allowing the “allow_duplicate: true” parameter to be processed is creating the issue of creating duplicates without restriction. When in reality, the app is designed and intended to ask and verify the user to check for duplicate data creation.

What to do: Remove `allow_duplicate` from any endpoint regular users or forms call. Decide the duplicate rule entirely on the server, never based on what the client sends. A flag the client controls can never be trusted to enforce a security rule. Only the database or the server can safely decide "this is a duplicate, don't create it."

If an internal or admin tool genuinely needs to force a duplicate on purpose, that should be its own separate, authenticated, logged admin action, not a flag any public API call can set. I would personally suggest you briefly discuss this with the designers or even the customers if they require duplicate data to be allowed for their business. And if there is not a strict requirement for it, it can be eliminated entirely.

Step 2: Give every "create" request a one-time ticket (idempotency key)

What this means: Before the browser sends a "create lead" request, it generates a random one-time code, like a ticket number, and attaches it. The server remembers which ticket numbers it has already seen.

What to do:

- When the form is submitted, generate a random ID on the frontend: `crypto.randomUUID()`.
- Send it in a header, e.g. `Idempotency-Key: <that random ID>`.
- On the server, before creating anything, check whether that ticket number has been seen. Seen before –> return the lead that already exists, don't create a new one. Not seen –> create the lead and remember the ticket number for about a day.

Why this matters: This one mechanism protects against every kind of duplicate — double-clicks, flaky connections causing retries, and scripted replay attacks — all in one place. It doesn't rely on anyone telling the truth about their intentions.

Step 3: Add a request limit (rate limiting)

What this means: Set a maximum number of requests any one user, IP, or account can send to this endpoint in a given time window, e.g. ~30 per minute. The Final rate limit values should be determined after discussion with the product team or relevant administration. Please confirm the thresholds with them before implementation.

What to do:

- Add a rate-limit check that runs before the request reaches the lead-creation logic. For a Next.js app on Vercel, the standard tool is **Upstash Ratelimit**, a small library built for exactly this.

- A basic rule looks like one line: `Ratelimit.slidingWindow(10, "60 s")`, meaning 10 requests per 60 seconds. You can check out maybe a youtube tutorial for this as well. It is a standard framework so the functionality is the same across all environments.

- Identify the requester by their logged-in user ID when possible, not just their IP, since multiple legitimate users can share an IP, like people in the same office. As the app uses randomly generated user ID strings, I am assuming the variable will be used instead of static values.

- When someone goes over the limit, you have two options, you either reject the request by replying with a '429 - Too many requests' status or you could add a time delay for the request to be processed by let's say 10 seconds.

- Amazon has restrictive measures implemented built-in on its vercel servers which protects the product from dangerous DDoS attacks hence one big problem is already solved by default. And for a DoS prevention, since the application is hosted on Vercel so you just have to configure their WAF according to your requirement.

- Open the Firewall Dashboard, Go to the Vercel dashboard, select the Crusource CRM, and open the Firewall section in the sidebar –> Create a New Rule –> Click Configure (top right) –> New Rule –> Set the Condition In the "**IF**" section, add a condition that goes like this:

  [Filter: **Request Path**] [Operator: **Equals**] [Value: **/api/v1/leads**] [Combining operator: **AND**] [Filter: **Method**] [Operator: **Equals**] [Value: **POST**] [Action: **Rate Limit**] [Limiting Strategy: **Fixed Window**] [Time Window: **60 seconds**] [Request Limit: **30 requests (or whatever the product team approves)**] [Key: **User ID**] [Response: **Deny → 429 Too Many Requests**]

- Save and Publish –> Click Save Rule. Then click Review Changes at the top right and Publish. The rules take effect immediately no redeployment needed. Please notice that you should apply this rule for multiple POST endpoints For example, if I want 3 endpoints to include in the rate limiting logic: **/api/v1/leads** , **/api/v1/deals** , **/api/v1/accounts** then the WAF rule can go like this:

  [Filter: **Request Path**] [Operator: **Equals**] [Value: **/api/v1/leads**] [Combining operator: **OR**] [Value: **/api/v1/deals**] [Combining operator: **OR**] [Value: **/api/v1/accounts**] [Combining operator: **AND**] [Filter: **Method**] [Operator: **Equals**] [Value: **POST**] [Action: **Rate Limit**] [Limiting Strategy: **Fixed Window**] [Time Window: **60 seconds**] [Request Limit: **30 requests (or whatever the product team approves)**] [Key: **User ID**] [Response: **Deny → 429 Too Many Requests**]

Step 4: Add a backup safety net in the database

Even after Steps 1 – 3, it's good practice to have one more layer that physically prevents exact duplicates from being stored, in case of a bug anywhere upstream.

What to do: Add a database rule, a "unique constraint," that blocks two leads with the same email, name, and company from being saved within a short time window of each other.

Why? This doesn't replace Steps 1 – 3, it's a last-resort safety net in case something upstream fails. If one layer breaks, this one still catches the problem.

Suggested Order to Tackle These

1. Step 1 – remove the client-controlled duplicate bypass (closes the actual security hole)
2. Step 2 – add the idempotency ticket system (closes duplicate creation at the root)
3. Step 3 – add rate limiting (closes the volume/abuse angle)
4. Step 4 – add the database safety net (extra protection)

How to Confirm the Fix Worked

Once these changes are live, repeat the same test: send 50 identical requests back-to-back.

- **Rate limiting works** if, after a certain number of requests, you start getting `429 Too Many Requests` instead of `201 Created`.
- **The idempotency fix works** if sending the exact same ticket number twice returns the same lead both times, never a new one, even if `allow_duplicate: true` is still in the request body.
- **The duplicate-bypass issue is fully fixed** if removing the `allow_duplicate` field from the request changes nothing — it should have no effect either way, because the server should never have been listening to it in the first place.

One More Thing Worth Checking

This test only covered the Leads endpoint. The same two problems – no rate limit, a duplicate check that might trust the client may be present on other "create" endpoints too: Deals, Contacts, Tasks, and CSV bulk import. It's worth running the same check across all of them rather than fixing Leads in isolation.

Side Note – I have performed tests through the CSV importing and the CSV template has some minor fixes to be done As many column titles have split into two columns when they are supposed to contain a single value. It is not causing any issues at the server end but at the customer end, it can be confusing. I will include the descriptive report with photos/videos of any major findings I discover from my side.